

Embeddings

Lesson 4: Sequence-to-Sequence — Encoder–Decoder Translation with Attention

Agenda

- The seq2seq problem
- The encoder
- The decoder
- The encoder–decoder pipeline
- Encoder–decoder attention

Learning Objectives

By the end of this lesson, you'll be able to:

- Explain why translation needs an **encoder–decoder** architecture instead of the many-to-one encoder of lesson 2.

Why this matters

Lesson 2 built encoders that turn a sequence into **one vector**. That's enough to *classify* or *retrieve* — but not to **produce language**.

Translation is the honest stress test: the output is itself a sequence, of a **different length**, in a **different vocabulary**, with a **different word order**.

Every generative system you'll meet later — translation, summarization, chat — is this same shape: read a sequence, then write a sequence.

Part I — The Sequence-to-Sequence Problem

The lengths don't match

Translate the two English words **Let 's go** and you get **one** Spanish word: **Vamos**.

	English (source)	Spanish (target)
sentence	Let 's go	Vamos
tokens	2	1

Nothing lines up: $T_x \neq T_y$, the vocabularies are different, and even when the counts *do* match, the **order** often doesn't (**paper lantern** → **lámpara de papel**).

⚠ Lesson 2's many-to-one encoder gives one vector; the aligned many-to-many model needs $T_x = T_y$. **Neither shape fits translation.**

The vocabulary: tokens plus special symbols

Networks don't read words, they read vectors (lesson 1). Two vocabularies — one per language — each with a few **special tokens** on top of the real words:

Token	Role
<SOS>	start of sentence — the decoder's first input, the "begin translating" signal
<EOS>	end of sentence — the decoder emits it to declare the translation finished
<PAD>	filler so a batch of different-length sentences forms one rectangular tensor
<UNK>	any word outside the vocabulary

Each token id goes through its own **embedding layer** — one for the source language, one for the target.

Part II — The Encoder

Reading the source sentence

The encoder is **exactly** the LSTM encoder from lesson 2 — nothing new. Feed the source tokens in order; at each step the cell updates its hidden state h_t and cell state c_t :

$$h_t, c_t = \text{LSTM}(x_t, h_{t-1}, c_{t-1}) \quad (1)$$

Unrolling reuses one block. The same weights and biases act at every position — which is why a single encoder handles sentences of *any* length without growing.

Parameters are tied across time: sentence length changes the *number of steps*, never the *number of parameters*.

The context vector

After the **last** source token, the encoder's final states are handed to the decoder. Together they are the **context vector**:

$$\text{context} = (h_{T_x}, c_{T_x}) \quad (2)$$

That pair is the encoder's entire output — its attempt to compress **everything** the source sentence means into a fixed-size summary.

⚠ Note the word **fixed-size**: whether the source is 3 words or 40, the context vector has the same dimensions. File that — it becomes Part V's problem.

Part III — The Decoder

Starting from the context vector

The decoder is **another LSTM**, with its own weights and its own (Spanish) embedding layer. Two things get it going:

1. Its hidden and cell states — call them s_i , c_i^{dec} , using s rather than h so the decoder's own states never get confused with the encoder's h_j once both are in play (Part V) — are **initialised** with the encoder's context vector (eq. 2): $s_0 = h_{T_x}$. It starts already "knowing" the source sentence.
2. Its first *input token* is **<S0S>** — the signal to begin producing words.

The encoder and decoder are **separate networks with separate weights**. The only thing that crosses between them is the context vector.

The output head: linear + softmax

The decoder's LSTM output at step i is a d_h -dimensional vector. To turn it into a *word*, project it onto the target vocabulary and normalise:

$$\hat{y}_i = \text{softmax}(W_o s_i + b_o) \in \mathbb{R}^{|V_{\text{es}}|} \quad (3)$$

- W_o has **one output per Spanish vocabulary entry** — 5,000 words means 5,000 numbers.
- Softmax turns those scores into a **probability distribution** over the whole Spanish vocabulary.
- Picking the highest-probability entry (**greedy decoding**) gives the word for this step.

The generation loop

The decoder writes **one word per step**, feeding its own output back in:

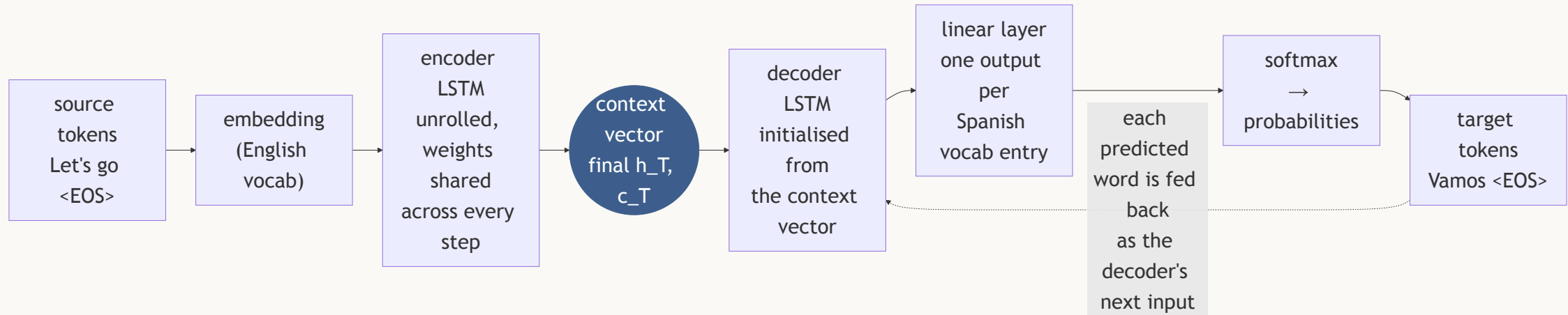
- **Step 1** — input `<SOS>` → predicts `Vamos`
- **Step 2** — input `Vamos` → predicts `<EOS>`
- **Stop** — `<EOS>` emitted, so the translation is `Vamos`

This is **autoregressive** generation: each prediction becomes the next input, so the decoder decides *for itself* how long the output is. It stops when it emits `<EOS>` (or hits a safety length cap).

That self-terminating loop is what finally frees T_y from T_x — the model chooses the output length.

Part IV — Encoder–Decoder Pipeline

The whole pipeline, in a picture



Worked example: **Let 's go** → **Vamos**

Phase	Step	Input	State	Output
encode	1	Let 's	h_1, c_1	—
encode	2	go	$h_2, c_2 = \text{context}$	—
decode	1	<SOS>	init from context	Vamos
decode	2	Vamos	updated	<EOS>

Two encoding steps, two decoding steps, and a **1-word** translation of a **2-word** sentence — with nothing anywhere fixing the ratio.

Two ways to run the decoder

The decoder's input at step i can come from two places — and **training and inference differ**:

	Decoder input at step i	Used when
Teacher forcing	the correct word y_{i-1} from the reference translation	training
Free running	the model's own prediction $\hat{y}_{i-1}$	inference (and evaluation)

Teacher forcing trains faster and more stably: an early mistake doesn't derail the whole sentence, and every step gets a clean input.

⚠ The cost is **exposure bias**: the model never practises recovering from its own errors, then meets exactly that situation at inference. This is why a model can look excellent in training and drift at test time.

Classic seq2seq: strengths & limitations

-  Handles $T_x \neq T_y$ — the output length is decided by the model, not by the input.
-  Encoder and decoder are independent: different languages, different vocabularies, different sizes.
-  Parameter count is independent of sentence length (weight tying, eq. 1).
-  Everything the decoder will ever know about the source is squeezed through **one fixed-size context vector**.
-  Still **sequential** — no parallelism across time, in either half.

Part V — Encoder–Decoder Attention

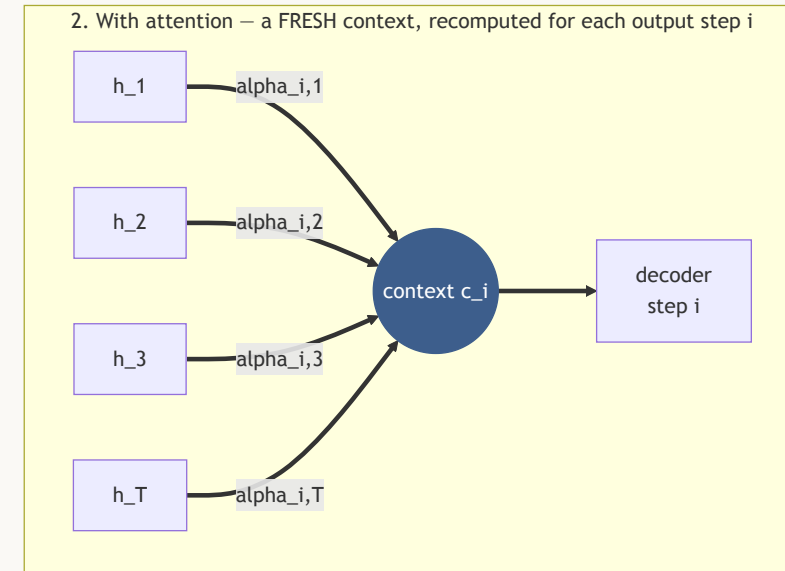
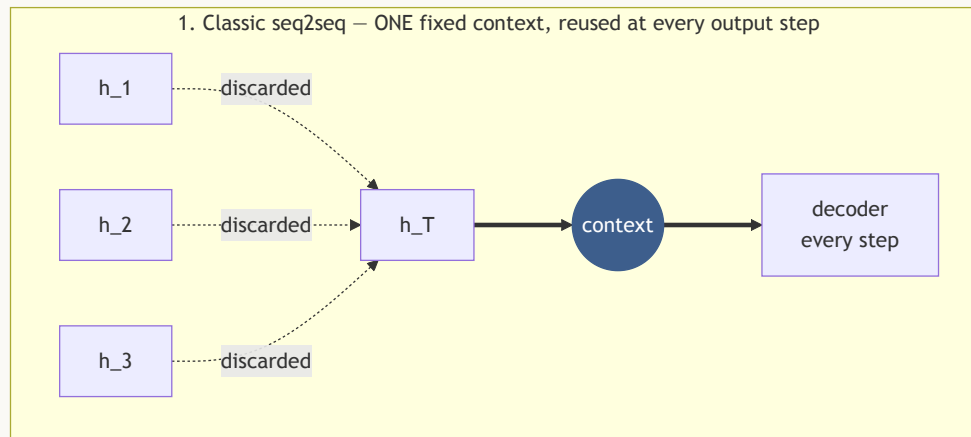
The fix: keep every encoder state

Classic seq2seq **throws away** $h_1, \dots, h_{T_x-1}$ and keeps only the last one. Attention's move is almost embarrassingly simple:

Keep them all. Then, at every decoding step, let the decoder build a *fresh* summary of the source — weighted toward whichever words matter for the word it is writing *right now*.

Instead of one context vector reused T_y times, there are now T_y context vectors, each computed on demand.

Bottleneck vs. attention, in a picture



Query, Key, Value — the three roles

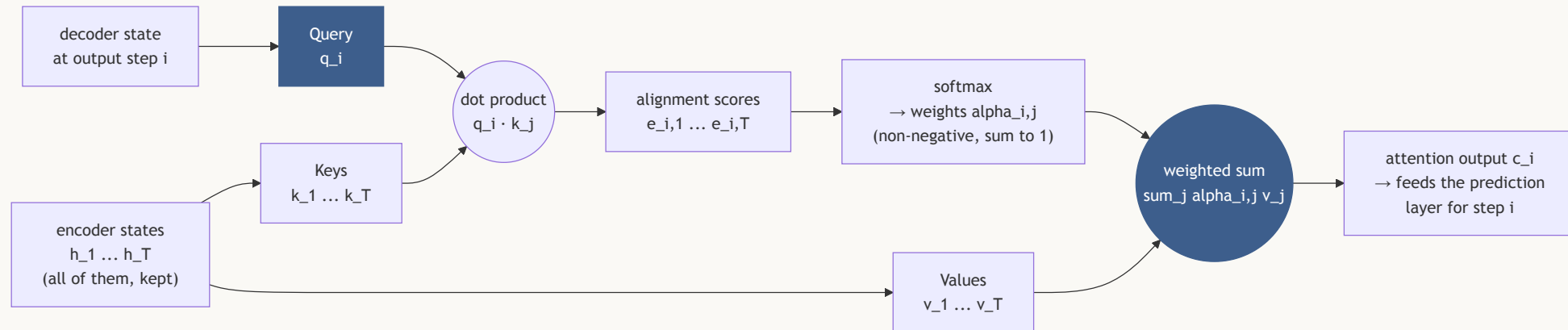
To decide where to look, attention gives every vector one of three jobs:

Role	Computed from	Reads as
Query q_i	the decoder state at step i	"what am I trying to write right now?"
Key k_j	each encoder state h_j	"what does this source word offer?"
Value v_j	each encoder state h_j	"what do I hand over if I'm selected?"

Each is a learned linear projection: $q_i = W_Q s_i$, $k_j = W_K h_j$, $v_j = W_V h_j$.

Why separate k and v ? The features that make a word *easy to find* need not be the features worth *passing along* — separating them lets the model learn both independently.

The QKV dataflow, in a picture



The math, step by step

$$e_{i,j} = q_i^\top k_j, \quad \alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{j'=1}^{T_x} \exp(e_{i,j'})}, \quad c_i = \sum_{j=1}^{T_x} \alpha_{i,j} v_j \quad (4)$$

1. **Score** every source position against the current query — a plain **dot product**, one number per source word.
2. **Normalise** with softmax $\rightarrow$ weights $\alpha_{i,j}$ that are non-negative and sum to **1** across the source.
3. **Blend** the values with those weights $\rightarrow$ the context vector c_i for *this* output step.

Read $\alpha_{i,j}$ as a percentage: *"while writing output word i , the model spent $\alpha_{i,j}$ of its attention on source word j ."*

What attention actually bought us

- **No bottleneck.** The decoder reaches any source word directly, whatever the sentence length.
- **A shorter gradient path.** As in lesson 2: the loss reaches h_j in *one hop* (a weight), not through a product of Jacobians — so long sentences stay learnable.
- **A window into the model.** The α matrix is a readable, plottable alignment.

⚠ **Misconception:** "attention replaces the LSTM here." It does **not** — both encoder and decoder are still LSTMs, still sequential. Attention adds a *lookup* across them. Removing recurrence entirely is Module 2's move.

Attention in seq2seq: strengths & limitations

- ✓ Fixes the bottleneck without touching the encoder or decoder architecture — it's an add-on.
- ✓ Quality gains grow **with sentence length** — exactly where classic seq2seq degrades.
- ✓ Gives interpretable alignments almost for free.
- ⚠ Costs $T_x \times T_y$ score computations per sentence, plus keeping all encoder states in memory.
- ⚠ Attention weights show *where the model looked* — suggestive, but **not** a guaranteed causal explanation.

→ To the homework lab

The homework hands you a **working English→Spanish translator, trained in a few minutes** — take it apart. Two models, identical except: **one has attention, one doesn't**.

- **C1 — does attention actually help?** Compare both models' translations and BLEU.
- **C2 — read the heatmap.** Plot the α matrix and check whether alignments make linguistic sense.
- **C3 — where does the bottleneck bite?** Break results down by sentence length.
- **C4 — teacher forcing.** Measure the training/inference mismatch you just met.
- **C5 — decoding.** Probe how the greedy loop chooses, and where it goes wrong.
- **C6 — synthesis**, including a comparison against a **real** pretrained translator.

Six guided experiments, then an open-ended assignment. Turn the knobs yourself — more data, subword tokenization, pretrained embeddings, deeper stacks — and see what actually moves the needle. The code is given and runs; the analysis and experimentation are yours.

What's next

You now have every piece of the modern architecture **except one**. The encoder–decoder skeleton, the autoregressive decoding loop, and Query/Key/Value attention all carry over unchanged into **Module 2 — Transformers**.

The one change: the Transformer **deletes the recurrence**. No LSTM, no left-to-right state passing — attention alone, computed for every position **in parallel**.

Curious now? Bahdanau, Cho & Bengio (2015) introduced exactly this attention mechanism, and Sutskever, Vinyals & Le (2014) is the seq2seq architecture in its original form.

References & further reading (1/2)

- [1] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Proc. NeurIPS*, 2014 (arXiv:1409.3215).
- [2] K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in *Proc. EMNLP*, 2014 (arXiv:1406.1078).
- [3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *Proc. ICLR*, 2015 (arXiv:1409.0473) — the attention mechanism of eq. (4).
- [4] M.-T. Luong, H. Pham, and C. D. Manning, "Effective Approaches to Attention-based Neural Machine Translation," in *Proc. EMNLP*, 2015 (arXiv:1508.04025) — the dot-product scoring used here and in the lab.

References & further reading (2/2)

- [5] A. Vaswani et al., "Attention Is All You Need," in *Proc. NeurIPS*, 2017 (arXiv:1706.03762) — where Module 2 picks up.
- [6] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997.
- [7] J. Starmer, "Sequence-to-Sequence Encoder-Decoder Neural Networks" and "Attention for Neural Networks," *StatQuest* — the video walkthroughs this lesson's narrative and **Let's go** example follow.